

Structures and Unions

Unions in C

- ▶ A union is a variable that may hold objects of different types and sizes.
- ▶ Compiler tracks size and alignment requirements.
- ▶ Provides a way to manipulate different kinds of data in a single area of storage.
- ▶ No machine-dependent information is embedded in the program.
- ▶ Analogous to variant records in Pascal.

Example Union Declaration

```
union u_tag {  
    int ival;  
    float fval;  
    char *sval;  
} u;
```

- ▶ The variable `u` can hold an `int`, `float`, or `char *` at different times.
- ▶ Size of `u` is large enough to hold the largest of the three types.

Example Usage of Union

If the variable `utype` is used to keep track of the current type stored in `u`, then

```
if (utype == INT)
    printf("%d\n", u.ival);
else if (utype == FLOAT)
    printf("%f\n", u.fval);
else if (utype == STRING)
    printf("%s\n", u.sval);
else
    printf("bad type %d in utype\n", utype);
```

- ▶ Programmers need to keep track of the current type stored in the union.
- ▶ Results are implementation-dependent if accessed as wrong type.

Unions within Structures

```
struct {  
    char *name;  
    int flags;  
    int utype;  
    union {  
        int ival;  
        float fval;  
        char *sval;  
    } u;  
} symtab[NSYM];
```

- ▶ Unions may occur within structures.
- ▶ Accessing a member of a union in a structure follows the same notation as nested structures; `symtab[i].u.ival`, `symtab[i].u.sval[0]`.

Unions

- ▶ Unions provide a way to store different types of data in the same storage.
- ▶ Syntax is similar to structures, members accessed as `union-name.member`.
- ▶ Unions may occur within structures and arrays, and vice versa.
- ▶ Initialization is limited to the type of the first member.
- ▶ Useful for manipulating different types of data without embedding machine-dependent information.
- ▶ A union is a structure in which all members have offset zero from the base, the structure is big enough to hold the “widest” member, and the alignment is appropriate for all of the types in the union.

Bit-Fields

Bit-Fields in C

- ▶ A bit-field, or field, is a set of adjacent bits within a single implementation-defined storage unit that we will call a “word.”
- ▶ Bit-fields are useful when storage space is at a premium.
- ▶ Commonly used for packing several objects into a single machine word.
- ▶ One application is for single-bit flags in symbol tables.
- ▶ Allows direct definition and access of fields within a word.
- ▶ Syntax: `struct { unsigned int field_name : width; } variable_name;`

Bit-Fields Example

```
struct {  
    unsigned int is_keyword : 1;  
    unsigned int is_extern : 1;  
    unsigned int is_static : 1;  
} flags;
```

- ▶ Defines a variable `flags` with three 1-bit fields: `is_keyword`, `is_extern`, `is_static`.
- ▶ Field width is specified after the colon.

Bit-Fields Example (Contd.)

```
flags.is_extern = flags.is_static = 1;
```

- Turns on the `is_extern` and `is_static` bits.

```
flags.is_extern = flags.is_static = 0;
```

- Turns off the `is_extern` and `is_static` bits.

```
if (flags.is_extern == 0 && flags.is_static == 0)
```

- Tests if both `is_extern` and `is_static` bits are off.

Bit fields

- ▶ Fields can directly represent flags or small integers.
- ▶ Implementation-dependent: order, word boundary overlap, and alignment.
- ▶ Not portable for externally-defined data structures: big-endian, little-endian.
- ▶ Fields may be declared only as `ints`; specify signed or unsigned explicitly.
- ▶ They are not arrays and they do not have addresses, so the `&` operator cannot be applied on them.

Command-line Arguments

Command-line Arguments

- ▶ In C environments, there is a way to pass command-line arguments to a program.
- ▶ `main` is called with `argc` (argument count) and `argv` (argument vector).
- ▶ `argc` is the number of arguments, and `argv` is a pointer to an array of argument strings.
- ▶ Multiple levels of pointers are used to manipulate these strings.

Command-line Arguments

- ▶ echo program, which echoes its command-line arguments on a single line, separated by blanks.
- ▶ "echo hello, world" prints "hello, world".
- ▶ By convention, `argv[0]` is the name by which the program was invoked, so `argc` is at least 1.
- ▶ Here `argc` is 3, and `argv[0]`, `argv[1]`, and `argv[2]` are "echo" , "hello," , and "world" respectively.
- ▶ The standard requires that `argv[argc]` be a null pointer.

Echo Program - Version 1

```
#include <stdio.h>
/* echo command-line arguments; 1st version */
main(int argc, char *argv[])
{
    int i;
    for (i = 1; i < argc; i++)
        printf("%s%s", argv[i], (i < argc-1) ? " " : "");
    printf("\n");
    return 0;
}
```

Echo Program - Version 2

```
#include <stdio.h>
/* echo command-line arguments; 2nd version */
main(int argc, char *argv[])
{
    while (--argc > 0)
        printf("%s%s", *++argv, (argc > 1) ? " " : "");
    printf("\n");
    return 0;
}
```


Enhancements - Pattern-finding Program

- ▶ Pattern-finding program.
- ▶ The pattern to be matched is specified by the first argument on the command line.
- ▶ Optional flags for pattern inversion (-x) and line numbering (-n).

Pattern-finding Program - Code

```
#include <stdio.h>
#include <string.h>
#define MAXLINE 1000
int getline(char *line, int max);

main(int argc, char *argv[])
{
    char line[MAXLINE];
    long lineno = 0;
    int c, except = 0, number = 0, found = 0;
```

Pattern-finding Program - Code

```
while (--argc > 0 && (*++argv)[0] == '-')
    /* while arguments remain and the next begins with '-' */
    while (c = *++argv[0])
        switch (c) {
            case 'x':
                except = 1; break;
            case 'n':
                number = 1; break;
            default:
                printf("find: illegal option %c\n", c);
                argc = 0; found = -1; break;
        }
```

Pattern-finding Program - Code

```
if (argc != 1)
    printf("Usage: find -x -n pattern\n");
else
    while (getline(line, MAXLINE) > 0) {
        lineno++;
        if ((strstr(line, *argv) != NULL) != except) {
            if (number)
                printf("%ld:", lineno);
            printf("%s", line);
            found++;
        }
    }
    return found;
}
```

Walk through

- ▶ Suppose "find -xn abc" is the command-line.
- ▶ Suppose "find" is stored in locations 64-68, "-xn" in 72-75, "abc" in 80-83.
- ▶ Then argc is 3; argv[0], argv[1], argv[2] and argv[3] are 64, 72, 80 and 0 respectively.
- ▶ Suppose argv[0], argv[1], argv[2] and argv[3] are in locations starting 120, 124, 128, 132. So, argv is 120.
- ▶ The outer while loop sets argc to 2; ++argv sets argv to 124; * of that gives 72; [0] of that is '-'
- ▶ The inner while loop applies ++ on argv[0]; The 72 in 124 becomes 73; * of that gives 'x'; c is set to 'x'; except is set to 1.

Walk through

- ▶ The inner while loop applies ++ on argv[0]; The 73 in 124 becomes 74; * of that gives 'n'; c is set to 'n'; number is set to 1.
- ▶ The inner while loop applies ++ on argv[0]; The 74 in 124 becomes 75; * of that gives 0; c is set to 0; loop terminates.
- ▶ The outer while loop sets argc to 1; ++argv sets argv to 128; * of that gives 80; [0] of that is 'a'; loop terminates.
- ▶ The code proceeds to get lines.